Task 07: DDPM for MNIST Digit Generation

Akram Aki

June 23, 2026

1 Introduction

The goal of this task was to train a Denoising Diffusion Probabilistic Model (DDPM) that generates MNIST-like handwritten digit images. This continues the generative modelling tasks from the previous assignment, but the training principle is different from a GAN. In a GAN, a generator and discriminator are trained against each other: the generator tries to create realistic samples, while the discriminator tries to separate real and generated images. This adversarial setup can produce sharp samples, but training can become unstable if one model becomes too strong.
A diffusion model instead uses a fixed noise process and a learned denoising process. During the forward process, Gaussian noise is gradually added to real images until almost only noise remains. During training, a neural network learns to predict the noise that was added at a randomly chosen timestep. During sampling, the model starts from random noise and repeatedly applies the learned denoising steps to obtain a generated image. This makes the objective more direct than in a GAN, since the model is optimized with a noise-prediction loss instead of an adversarial loss.

2 Method

MNIST images were loaded with `torchvision` and transformed to tensors with shape $1 \times 28 \times 28$. The pixel values were kept in the range $[0, 1]$, as required for the task. The diffusion implementation used the `denoising-diffusion-pytorch` package with automatic normalization disabled, so the input tensors were not internally remapped to $[-1, 1]$.
For a clean image x_0, the forward diffusion process generates a noisy image x_t through

$$x_t = \sqrt{\bar{\alpha}_t} x_0 + \sqrt{1 - \bar{\alpha}_t}\epsilon, \qquad \epsilon \sim \mathcal{N}(0, I),$$

where $\bar{\alpha}_t$ is the cumulative product of the noise schedule. As t increases, the clean image contributes less and the Gaussian noise term becomes dominant.

The denoising model was a U-Net. For each batch, a random timestep was sampled and the model predicted the noise component $\epsilon_\theta(x_t, t)$. The training objective was mean squared error between the true and predicted noise,

$$\mathcal{L} = \|\epsilon - \epsilon_\theta(x_t, t)\|_2^2 .$$

The U-Net is useful for image generation because it combines downsampling and upsampling layers with skip connections. The downsampling path extracts larger-scale image features, while the skip connections preserve local spatial information that is important for digit strokes. The timestep input tells the network how much noise is present, so the same model can denoise images at all diffusion stages.
The first code test used a deliberately capped training run with `max_batches_per_epoch = 200`. This cap was used to verify that the implementation, checkpointing, sampling, and plotting worked before starting a longer run. The final report run used the full MNIST training loader and the configuration summarized in Table 1. Training was performed on my own PC with an NVIDIA RTX 3060 Ti and took approximately 49.3 minutes.

Quantity	Value
Image shape	$1 \times 28 \times 28$
Pixel range	$[0, 1]$
Batch size	128
Batches per epoch	469
Epochs	50
Optimizer	AdamW
Learning rate	$4 \cdot 10^{-4}$
U-Net dimensions	`dim = 32`, `dim_mults = (1, 2, 4)`
Trainable parameters	$2.61 \cdot 10^6$
Training diffusion steps	$T = 1000$
Sampling steps	250
Beta schedule	linear
Objective	noise prediction

Table 1: Training and sampling configuration used for the DDPM report run.

The training diffusion process used 1000 possible noise levels. For image generation, DDIM sampling with 250 reverse steps was used, which is faster than sampling through all 1000 reverse steps.

3 Results

Figure 1 shows real MNIST samples from the training set. These samples define the target image distribution that the diffusion model should learn.

Real MNIST Samples

Figure 1: Real MNIST samples from the training set.

Figure 2 shows the fixed forward diffusion process. The digit structure is gradually destroyed as the timestep increases, until the images are dominated by noise.

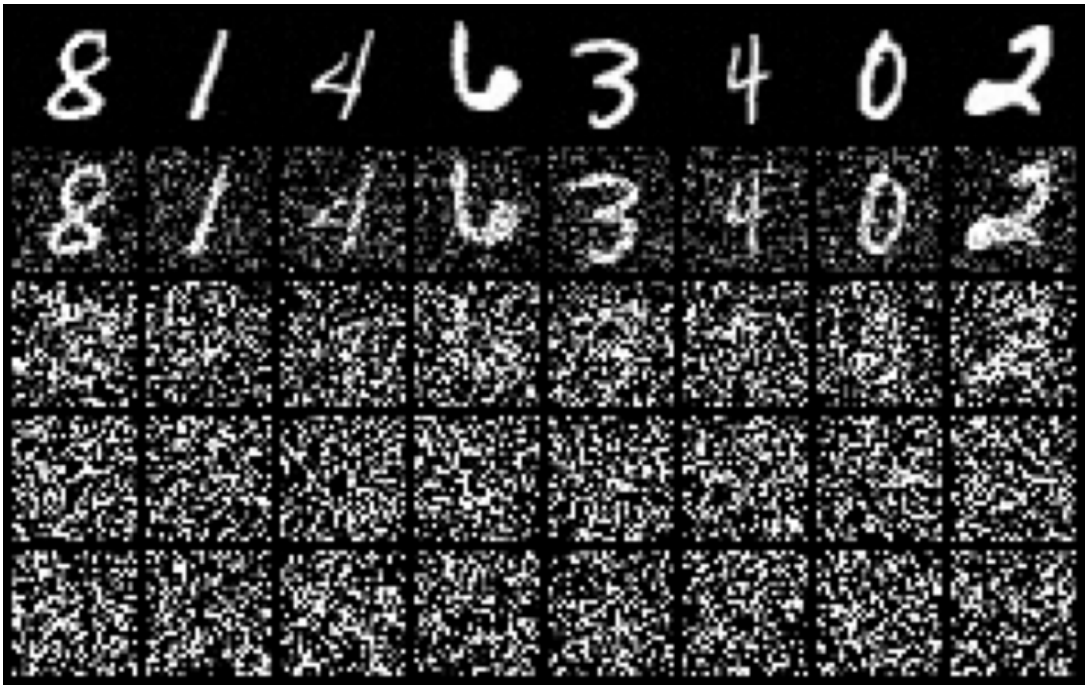


Figure 2: Forward diffusion snapshots for MNIST images. The forward process is fixed and gradually transforms clean data into noisy samples.

Figure 3 shows the noise-prediction loss during training. The first epoch had a loss of 0.0519, which was much larger than the later values and is therefore excluded from the plot to make the remaining trend easier to see. From epoch 2 onward, the loss decreased from 0.0205 to 0.0139, indicating that the U-Net learned to predict the added noise more accurately. The loss is useful for checking that training is stable, but visual samples are still needed to judge generation quality.

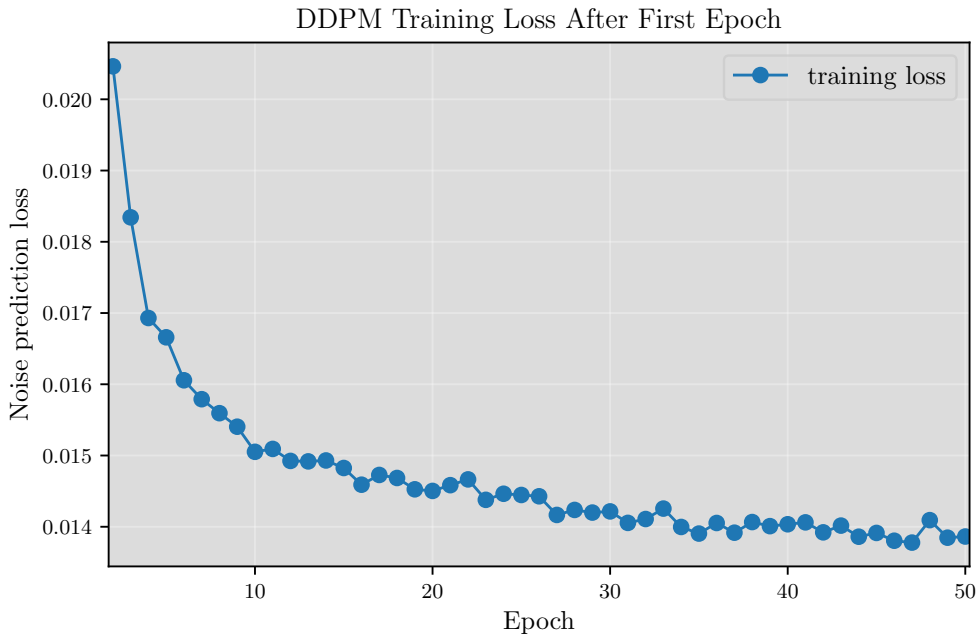
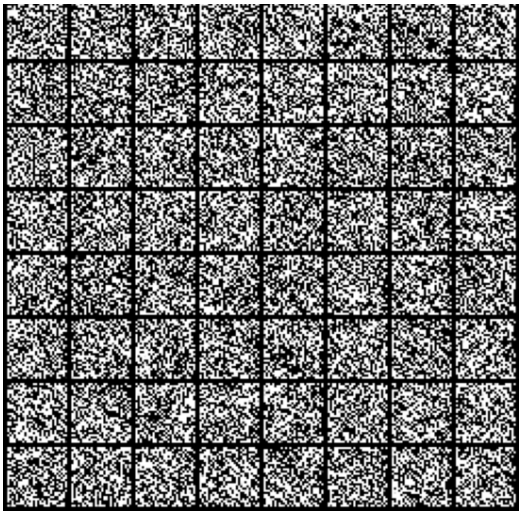


Figure 3: DDPM training loss from epoch 2 onward. The first epoch loss was 0.0519 and is excluded from the plot so that the smaller later changes are visible.

Figure 4 shows generated samples during training. At epoch 0, the model has random weights and the samples do not resemble digits. As training progresses, the samples become more structured and begin to show digit-like strokes. The final image quality depends strongly on the training time, so these samples should be interpreted together with the number of epochs and batches used. The epoch 50 checkpoint was selected for the final samples because it produced the clearest overall digit shapes among the saved sample grids.



(a) Epoch 0.

(b) Epoch 10.

(c) Epoch 30.

(d) Epoch 50.

Figure 4: Generated DDPM samples during training. The sample quality improves as the denoising model learns the MNIST image distribution.

Figure 5 shows a fresh set of final generated samples. The samples were generated by starting from random Gaussian noise and applying the learned reverse diffusion process. This figure was generated from the final checkpoint using 250 DDIM sampling steps.

Figure 5: Final generated MNIST samples from the trained DDPM.

Figure 6 visualizes intermediate reverse diffusion steps. The first row starts close to random noise. During the reverse process, the samples gradually become less noisy and more image-like, ending in the final generated digits. This illustrates the main difference from a GAN generator: the image is not produced in one forward pass from a latent vector, but through an iterative denoising chain.

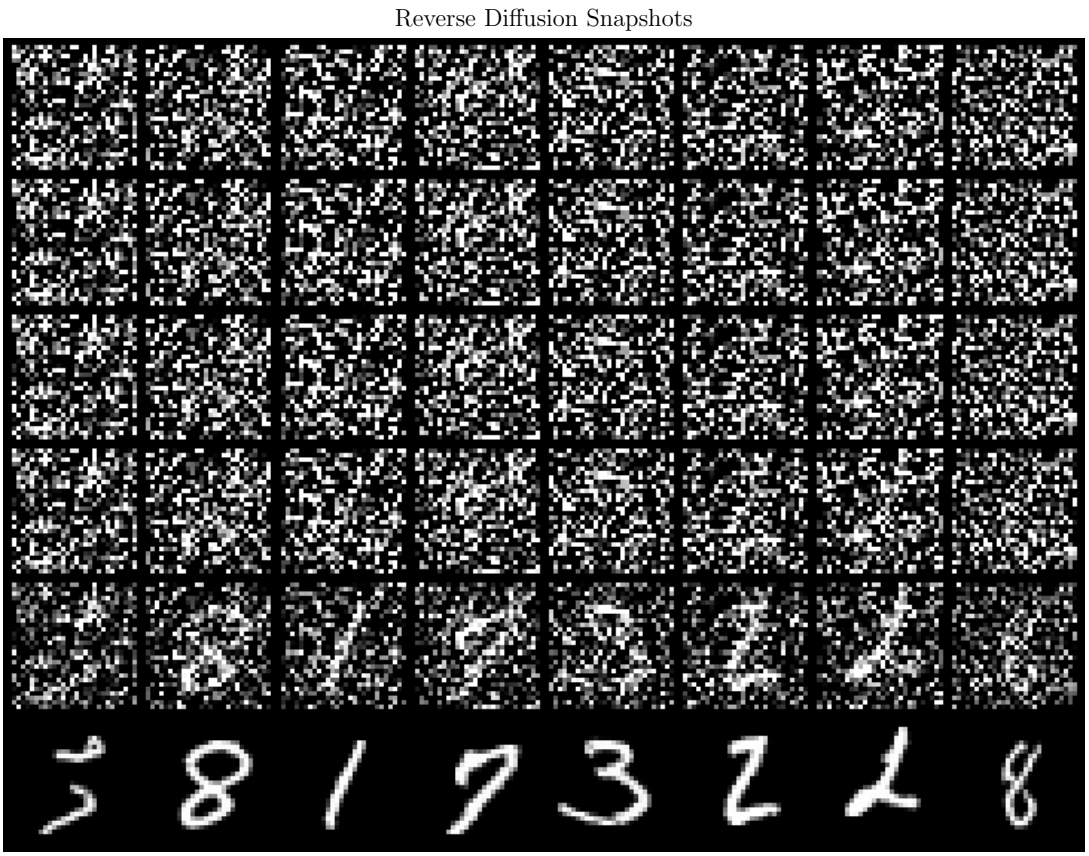


Figure 6: Snapshots of the learned reverse diffusion process. The model starts from random noise and iteratively denoises toward MNIST-like samples.

4 Conclusion

The DDPM implementation successfully trained a U-Net denoiser on MNIST and generated digit-like samples. The capped test run with 200 batches per epoch was useful for checking that the code worked and for estimating runtime. The longer final run produced the samples shown in this report and took approximately 49.3 minutes on a local RTX 3060 Ti system.

The main limitation is that diffusion models require many denoising evaluations during sampling, and image quality depends strongly on training time and model capacity. Possible improvements include training for more epochs, using a larger U-Net, testing a different beta schedule, using exponential moving average weights for sampling, or adding class conditioning to generate specific digits. All code used for data loading, training, sampling, and plotting is available in the accompanying GitHub repository.